

CHAINTRACK

Comprehensive Faculty Demonstration, Viva Defense & Presentation Coach

Repository: github.com/Om-Rajpure/ChainTrack
Verified Tests: 87/87 Passing (43 Contract + 44 Web/API)

Contract Coverage: 100% Statements, Funcs, Lines (92.4% Branch)
Deployment: Ethereum Sepolia (11155111) & Local Hardhat

1. Core Project Explanations by Duration

ChainTrack in One Sentence: A hybrid Web3 supply chain tracking system that anchors critical product custody and lifecycle events immutably on an Ethereum smart contract while maintaining searchable metadata off-chain in PostgreSQL, enabling zero-login public verification and mathematical tamper detection via cryptographic hashing.

30-Second Explanation (High Level)

"In traditional supply chains, product records are siloed in private corporate databases where they can be manipulated, forged, or disputed, allowing counterfeit goods to enter unnoticed. **ChainTrack solves this by separating proof from metadata.** Every handover—from Manufacturer to Distributor to Retailer—is authorized by role-based smart contracts and permanently anchored on Ethereum. Rich details live in PostgreSQL, and canonical metadata is hashed on-chain using Keccak-256. Any consumer can scan a QR code without logging in or holding crypto. The system re-hashes live database metadata and compares it against the blockchain proof to deliver an instant verdict: **Authentic, Data Mismatch, or Not Found.**"

2-Minute Explanation (Technical Overview)

"Good morning, Professor. Today I am presenting **ChainTrack**, an end-to-end blockchain supply chain traceability and anti-counterfeiting platform.

The Problem & Trust Deficit: In multi-tier supply chains, products move through independent stakeholders: Manufacturers, Distributors, Retailers, and Consumers. Centralized databases fail here because whoever owns the database server holds unilateral power to alter timestamps, forge transit checkpoints, or delete records retroactively.

Hybrid Storage Architecture: Storing images and descriptive metadata directly on Ethereum is cost-prohibitive. ChainTrack implements a dual-layer architecture: (1) An immutable **Solidity smart contract** (SupplyChain.sol) enforcing state transitions, actor roles, append-only history, and Keccak-256 data hashes; (2) An off-chain **Next.js + Prisma + PostgreSQL** layer for searchable metadata, dashboard analytics, and sessions; (3) An autonomous **background indexer** scanning logs to ensure eventual consistency.

Two-Step Custody Handover: A transfer is initiated by the current owner (InTransit), and only the designated active receiver can acceptTransfer or rejectTransfer (rolling custody back). Finally, the Retailer marks the product Sold.

Public Zero-Login Verification: Consumers scan the QR code to open /verify/[id]. The system dynamically recomputes the Keccak-256 hash from live database fields and verifies it against the blockchain proof, alerting users to any database tampering."

5-Minute Explanation (Engineering & Security Deep-Dive)

"Respected Guide, I would like to detail the engineering architecture, smart contract state machine, and tamper-detection mechanics of ChainTrack.

1. Motivation & Real-World Scope: Counterfeit products cause over \$450B in annual losses globally. Vulnerabilities occur at transit handovers. When systems rely on single SQL databases, insider threats or compromised servers alter custody histories undetected.

2. Strict State Machine: SupplyChain.sol manages 5 product states: Created → InTransit → AtDistributor → InTransit → AtRetailer → Sold. Handover uses an escrow protocol where transfers can be accepted or rejected with recorded on-chain reasons. Sold represents permanent terminal finality.

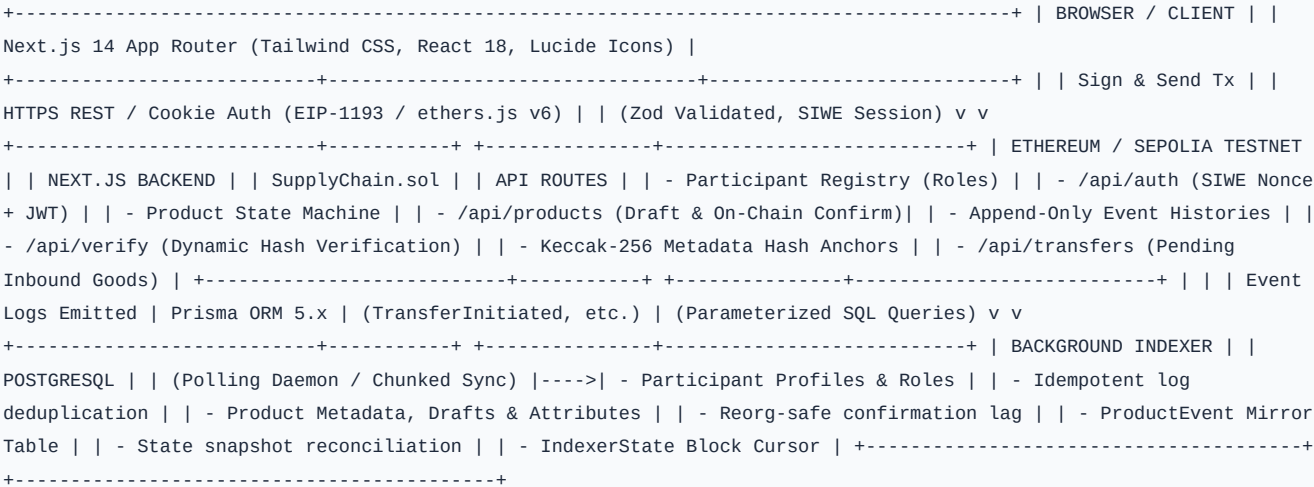
3. Canonical Metadata Hashing: We construct a deterministic JSON representation containing serial numbers, names, batch IDs, dates, and manufacturer wallets. Keys are sorted alphabetically, whitespace stripped, and hashed: DataHash = keccak256(CanonicalJSON). This 32-byte digest is anchored on-chain at registration.

4. Background Event Indexer: Direct RPC event scanning is slow and rate-limited. Our polling daemon tracks an atomic block cursor (IndexerState), queries logs in chunks, deduplicates via composite keys (txHash, logIndex), and takes on-chain state snapshots to mirror status into PostgreSQL.

5. Passwordless Security: Sign-In with Ethereum (EIP-4361) verifies single-use signed nonces, issuing stateless JWTs in httpOnly cookies. Smart contract modifiers (onlyOwnerOf, onlyActiveParticipant) enforce cryptographic authorization independently of the UI."

2. System Architecture & Component Interactions

High-Level Dual-Pipeline Architecture:



Why the Indexer Exists (Faculty Defense Point)

- 1. Relational Query Efficiency:** Ethereum is an execution trie, not a query engine. Filtering products by category, owner, or status via RPC requires scanning millions of historical logs, taking several seconds and hitting provider rate limits. The indexer enables sub-15ms SQL queries.
- 2. Resilient Event Mirroring:** The daemon tracks lastProcessedBlock in PostgreSQL and deduplicates events using a unique database constraint on (txHash, logIndex). Even across network crashes, no event is lost or duplicated.
- 3. Decoupled Verification:** Zero-login public verification reads directly from Ethereum RPC for ownership and history, maintaining availability even if the PostgreSQL database is temporarily offline.

3. Technology Stack & Selection Rationale

Technology	Exact Purpose in ChainTrack	Why Selected over Alternatives
Solidity (^0.8.24)	Core smart contract logic (SupplyChain.sol)	EVM standard; built-in overflow protection and custom errors (error NotOwner()) saving gas.
Hardhat	Local blockchain environment, testing, and deployment	Native TypeScript integration, instant local mining, gas profiling, and EVM compatibility.
Ethereum Sepolia	Public Proof-of-Stake test network	Official Ethereum testnet; mirrors mainnet consensus rules without real financial costs.
Next.js 14 (App Router)	Full-stack web application and backend API handlers	Unified TypeScript repo combining React Server Components with secure serverless API routes.
PostgreSQL + Prisma	Relational storage for off-chain metadata, search, and events	ACID compliance, parameterized queries, automated migrations, and multi-column filtering.
ethers.js (v6)	Client and server Web3 blockchain interactions	Modern, lightweight TypeScript library for transaction signing and Keccak-256 hashing.
SIWE + `jose` (JWT)	Cryptographic wallet authentication and sessions	EIP-4361 passwordless standard; signs nonces; JWTs sealed in httpOnly cookies.
Zod	Runtime API request and schema validation	Strict input sanitization before payloads reach database or hashing pipelines.
qrcode & html5-qrcode	QR code generation and in-browser camera scanning	Generates PNG QR tags; parses video streams client-side without native mobile app downloads.

4. Step-by-Step Live Demonstration Script

Follow this exact clicking sequence and speaking script during the live project evaluation:

Phase & Screen	Exact UI Actions	What to Say (Verbatim Presenter Script)
1. Public Landing /	Open homepage in browser. Point to the verification search bar and Connect Wallet button.	"This is ChainTrack's public portal. The platform separates public consumer verification from authenticated enterprise supply chain operations."
2. Manufacturer Login /login	Select Account #2 (Manufacturer) in MetaMask. Click Connect & Sign SIWE nonce.	"I am connecting as Acme Manufacturing. Notice there are no passwords—our server issues a cryptographic nonce that MetaMask signs to issue a secure session cookie."
3. Product Registration /products/new	Fill product form (Smart Health Tracker, BATCH-2026-X1). Submit & confirm MetaMask tx.	"When submitted, the backend creates a canonical JSON representation and hashes it with Keccak-256. MetaMask prompts registerProduct on-chain, anchoring this 32-byte digest immutably."
4. Custody Handover /products/1	Click Initiate Transfer . Choose SwiftLogistics (Distributor). Confirm tx. Status becomes IN_TRANSIT.	"We enforce a two-step escrow handover. The product is now locked in transit. Custody cannot be transferred to an unauthorized party or wrong role."
5. Distributor Acceptance /transfers	Switch MetaMask to Account #3 (Distributor) . Open Transfers, click Accept Transfer .	"As the Distributor, I inspect incoming packages. If goods were damaged, I could click 'Reject' with an on-chain reason. Everything is intact, so I accept, becoming the new owner."
6. Transit Checkpoint /products/1	Click Add Location Update . Enter 'Cold Vault 2', note 'Temp verified 4°C'. Confirm tx.	"The owner can log transit checkpoints without altering ownership, creating an immutable audit trail of environmental and routing updates."
7. Retailer Sale /products/1	Transfer to Retailer → Switch to Account #4 (Retailer) → Accept → Click Mark as Sold .	"The Retailer receives the unit and marks it 'Sold'. Once sold, the smart contract state machine locks permanently. No further transfers or edits can ever occur."
8. Public Verification /verify/1	Open Incognito Tab → Navigate to /verify/1 (or scan QR code).	"Notice I am completely logged out. Any consumer scanning the QR code receives an instant AUTHENTIC verdict, displaying the verified owner and complete blockchain event timeline."

5. The Tamper Detection Demonstration ("The Wow Moment")

Live Tamper-Detection Protocol (45 Seconds):

- 1. Initial State:** Open /verify/1 → Show the faculty the green **AUTHENTIC** banner.
- 2. Simulate Database Compromise:** Open Prisma Studio (or SQL terminal) → Modify Product #1's batchNumber from BATCH-2026-X1 to BATCH-FAKE-99.
- 3. Trigger Re-Verification:** Refresh /verify/1 in the browser.
- 4. Result:** The verdict immediately switches to **DATA MISMATCH (Amber Warning)**.
- 5. Explanation:** "The blockchain record never changed. The application dynamically regenerated the Keccak-256 hash from the modified database record, and detected that it no longer matched the hash permanently anchored on Ethereum during manufacturing."
- 6. Restore:** Revert batchNumber back to BATCH-2026-X1 → Refresh → Green **AUTHENTIC** is restored.

6. Comprehensive Viva Voce & Technical Defense (32 Questions)

Detailed answers across all 6 technical evaluation domains:

Domain 1: Problem Statement & Core Architecture

Q1: Why is blockchain required? Why not just use PostgreSQL with digital signatures?

A centralized database has a single administrator who can alter data, delete logs, or manipulate keys. Blockchain provides decentralized consensus and immutable state enforcement across competing supply chain participants without relying on a trusted central authority.

Q2: What exact data is stored on-chain vs. off-chain?

On-chain: product IDs, current owner and pending receiver addresses, status enum, append-only history entries (timestamp, location, note), participant roles, and the 32-byte Keccak-256 metadata hash. Off-chain (PostgreSQL): product names, descriptions, images, organization details, and searchable indexes.

Q3: Why is Ethereum Sepolia used instead of Ethereum Mainnet?

Sepolia is an official Ethereum Proof-of-Stake testnet mirroring mainnet EVM execution and consensus without incurring real financial gas costs, providing an authentic production-grade demonstration environment.

Q4: Why not store entire images and product descriptions directly on-chain?

Storing 1 MB of arbitrary data on Ethereum costs thousands of dollars in gas. ChainTrack uses a hybrid model: the smart contract holds only what is necessary for security, anchoring off-chain data integrity via cryptographic hashing.

Q5: What is the physical oracle problem, and how does ChainTrack handle it?

The oracle problem refers to bridging physical reality with digital records. In ChainTrack, unique serial UUIDs are bound to QR tags, and two-step custody transfers ensure that physical recipients verify goods before accepting on-chain liability.

Domain 2: Event Indexer & Database Architecture

Q6: Explain why the background Indexer is necessary.

Directly querying Ethereum event logs for complex frontend filters or search requires scanning thousands of blocks via RPC, which is slow and rate-limited. The indexer mirrors events into PostgreSQL for sub-millisecond querying while keeping the blockchain as the source of truth.

Q7: How does the indexer handle crash recovery and prevent duplicate events?

The indexer tracks its progress in `IndexerState.lastProcessedBlock`` and uses a database composite unique constraint on `(txHash, logIndex)``. On restart, it resumes from `lastProcessedBlock + 1``, and duplicate logs are skipped idempotently.

Q8: What happens if PostgreSQL goes completely offline?

Zero-login public verification (`/verify/[id]`) continues to function because it reads core ownership, status, and event history directly from the Ethereum smart contract via RPC, displaying on-chain data with a 'Details unavailable' banner.

Q9: How does the indexer guard against blockchain reorgs?

On Sepolia, the indexer applies a 2-block confirmation lag (reading up to `latest - 2``), ensuring that temporary chain reorganizations do not cause inconsistent state transitions to be indexed.

Q10: Why are `ProductEvent`` and `Product`` tables logically linked without database foreign keys?

To ensure the indexer can ingest and persist on-chain events immediately, even if the corresponding product draft record has not yet been confirmed or synced in PostgreSQL.

Domain 3: Smart Contract & State Machine

Q11: Explain the two-step transfer protocol in `SupplyChain.sol``.

To prevent sending inventory to wrong or unwilling parties, `initiateTransfer`` sets status to `InTransit`` and assigns a `pendingReceiver``. Only the designated receiver can invoke `acceptTransfer`` (claiming ownership) or `rejectTransfer`` (rolling status back to the sender).

Q12: What happens if a receiver rejects a transfer?

Custody remains with the original sender, `pendingReceiver`` is cleared, status rolls back (`Created`` if from Distributor, `AtDistributor`` if from Retailer), and a `TransferRejected`` event is emitted with the recorded rejection reason.

Q13: Why are custom Solidity errors used instead of `require(condition, 'string')`?

Custom errors (`error NotOwner()`) are significantly cheaper in gas because they compile to a 4-byte selector hash rather than storing expensive ASCII string literals in contract bytecode.

Q14: What prevents two products from registering with the same metadata hash?

The smart contract maintains a storage mapping `mapping(bytes32 => bool) private hashUsed``. If `hashUsed[dataHash]` is already true, `registerProduct`` immediately reverts with `DuplicateHash()`.

Q15: Can the Admin change product ownership or mark a product Sold?

No. The Admin role is strictly restricted to participant registration and deactivation. Ownership mutations can only be executed by the current cryptographic owner or designated receiver.

Q16: Can a product be transferred directly from Manufacturer to Retailer?

No. The contract verifies that from status `Created``, transfers can only be addressed to an active account with `Role.Distributor``. Skipping intermediary roles reverts with `InvalidReceiver()`.

Domain 4: Cryptographic Hashing & Verification

Q17: Why is canonical JSON serialization required before hashing?

JSON key ordering is non-deterministic across operating systems and runtimes. Sorting keys alphabetically and normalizing dates and addresses guarantees that identical metadata always produces the exact same UTF-8 byte stream and Keccak-256 hash.

Q18: Which fields are included in the canonical hash calculation?

`serialNumber`` (UUID per unit), `name``, `category``, `batchNumber``, `manufacturingDate``, `manufacturerWallet``, and sorted optional `attributes`` and `imageHash``. The serial number guarantees unique hashes even for identical units in the same batch.

Q19: How does the public verification endpoint detect off-chain metadata tampering?

It fetches the on-chain hash from `getProduct(id)``, fetches the database record, executes `computeProductHash()` over live database fields, and compares the two hashes. If unequal, it flags `DATA_MISMATCH``.

Q20: Why is `keccak256`` chosen over standard `SHA-256``?

`keccak256`` is the native cryptographic hash function of the Ethereum Virtual Machine (EVM), providing optimal gas efficiency and seamless compatibility across smart contracts and ethers.js.

Domain 5: Security, Authentication & Web3 Identity

Q21: Explain how Sign-In with Ethereum (SIWE) works in ChainTrack.

The server generates an expiring single-use cryptographic `nonce``. MetaMask signs this message. The server recovers the signer address, verifies their on-chain role via `getParticipant()`, and issues a stateless JWT stored in an `httpOnly, Secure, SameSite=Lax`` cookie.

Q22: If an attacker intercepts the JWT session cookie, can they alter blockchain state?

No. The JWT only authorizes API read/write operations (e.g. creating drafts). All state-changing blockchain transactions require an explicit private key signature directly inside MetaMask; the backend never possesses user private keys.

Q23: Why is role authorization enforced on the smart contract rather than just UI?

Frontend route masking and button hiding are purely for user experience. Anyone can craft a raw transaction and submit it to the contract address via RPC. Solidity modifiers (`onlyAdmin``, `onlyActiveParticipant``, `onlyOwnerOf``) enforce protocol-level security.

Q24: What happens if a participant is deactivated on-chain while currently logged in?

The smart contract immediately rejects any transaction from that wallet via `modifier onlyActiveParticipant``. API route middleware also checks on-chain active status for sensitive operations.

Q25: How are replay attacks prevented during wallet login?

Each login challenge includes a unique random nonce with a 5-minute expiration timestamp, recorded in the `AuthNonce` table and marked `usedAt` upon first verification.

Domain 6: Economics, Testing, Limits & Scalability

Q26: What is the gas cost profile of ChainTrack transactions?

Product registration costs ~120,000–160,000 gas; transfer initiation, acceptance, and marking as sold cost ~45,000–75,000 gas.

On EVM Layer-2 rollups (Arbitrum, Polygon, Base), transactions cost less than \$0.01 each.

Q27: What automated testing coverage was achieved?

87 total automated tests: 43 smart contract tests (100% statements, 100% functions, 100% lines, 92.4% branches) and 44 web/API integration tests passing cleanly with zero linter errors.

Q28: What are the primary technical limitations of this MVP?

(1) Physical-to-digital gap: Standard QR codes can be copied (production requires NFC crypto-tags); (2) Escrow timeout: If a receiver never responds, products stay `InTransit` (future extension: `cancelTransfer`); (3) Layer-1 throughput: High-volume scale requires Layer-2 rollups.

Q29: Can the smart contract be upgraded if business rules change?

In this MVP, bytecode is permanently immutable for trust. In an enterprise environment, an ERC-1967 Transparent or UUPS Proxy pattern would be used to upgrade logic contracts while preserving state.

Q30: How does the QR code scanner work without external hardware?

It uses the `html5-qrcode` library to capture video frames directly from mobile and desktop webcams, parsing QR code matrix data entirely client-side without third-party server uploads.

Q31: What prevents text overflow attacks on-chain (e.g., massive location strings)?

The contract enforces strict length limits: `MAX\_LOCATION\_LENGTH` is 100 bytes and `MAX\_NOTE\_LENGTH` is 280 bytes. Exceeding lengths reverts with `StringTooLong`.

Q32: Could this system support automated IoT sensor inputs?

Yes. IoT gateways equipped with secure hardware wallets can be registered with authorized roles to call `addLocationUpdate` or trigger automated escrow logic when sensor thresholds (e.g., temperature) are breached.

7. Day-Before Preparation Checklist & Demo Cheat Sheet

Pre-Demo Verification Checklist:

- [] **MetaMask Wallets:** 4 Accounts labeled (Admin, Manufacturer, Distributor, Retailer) with Sepolia test ETH.
- [] **Local Environment Backup:** Hardhat node tested (npm run chain) and seeded (npm run seed).
- [] **Test Suite Execution:** Run npm run test → Verify all 87 automated unit and integration tests pass.
- [] **Database Status:** PostgreSQL active and IndexerState initialized.
- [] **Browser Tabs Ready:** App (Tab 1), Public Verify (Tab 2), Etherscan (Tab 3), GitHub (Tab 4), Prisma Studio (Tab 5).

Final 7-Minute Presentation Sequence Runbook

Time Window	Presentation Topic	Key Talking Point & Screen Focus
0:00 - 0:45	Problem Statement	Siloed corporate databases enable undetected counterfeits and unprovable custody disputes.
0:45 - 1:30	Solution & Architecture	Hybrid model: Ethereum anchors state, roles, and Keccak-256 hash; PostgreSQL stores rich metadata.
1:30 - 2:00	Tech Stack	Solidity 0.8.24, Hardhat, Next.js 14, PostgreSQL, Prisma ORM, SIWE + JWT, ethers.js v6.
2:00 - 4:30	Live Lifecycle Demo	Manufacturer registers → InTransit → Distributor accepts → Checkpoint → Retailer marks SOLD.
4:30 - 5:30	Public Verification	Zero-login scan opens /verify/1 → Instant AUTHENTIC verdict and complete on-chain timeline.
5:30 - 6:15	Tamper Detection Demo	Modify database batch number in Prisma Studio → Refresh → Instant DATA MISMATCH warning.
6:15 - 7:00	Summary & Viva Transition	87/87 automated tests passing, 100% smart contract statement coverage. Open for viva questions.